ChatGPT

PolyHedg 10-Hour Hackathon Strategy

This plan breaks down each development step across a 10-hour timeline, balancing **core features** (market data, hedging simulation, basic UI) with speed and persuasion techniques. We leverage **Next.js** (fast React framework [1]), **Tailwind CSS** (rapid utility styling [2]), and deploy on **Cloudflare Pages** for free, instant global hosting with auto-deploy on each git push [3]. The demo uses real Polymarket data (via their REST API [4] [5]) and a fake user flow (no real auth needed).

Core goals: show a live prediction market (Polymarket) and a simple hedging tool, using actual market data to persuade the judge that our app is functional and informative. We cut non-essential features (no real wallet integration) and focus on quick wins and polish: clear UI, charts of probabilities, and trust cues.

Figure: Example coding environment. We set up a Next.js project quickly (using `create-next-app`) and install Tailwind CSS for fast styling [1] [2]. Leveraging code frameworks and AI-assisted coding (e.g. GitHub Copilot) speeds up development dramatically in hackathons [1] [6].

Step-by-Step Timeline

1. **0–15 min (Kickoff & Planning):** Gather the team and clarify *must-have* features: a market list page, a market detail page (with Yes/No odds and chart), and a hedging calculator. Quickly sketch the user flow (login → market selection → hedging). Assign roles (frontend, backend/API, design). **Mindset:** focus on the MVP. Hackathons work best with a tight focus on core functionality [7]. Note key Polymarket endpoints: e.g. `GET /markets` and `GET /trades` from their API [4] [5] to fetch market info and recent trades.

2. **15–30 min (Initialize Project):** Run `npx create-next-app polyhedg` for boilerplate. Set up a GitHub repo and connect Cloudflare Pages (choose Next.js static site). Install Tailwind CSS (utility-first styling for rapid layout [2]) and a UI library (e.g. Shadcn/UI components for ready-made React UI). Initialize the routing: create pages for `/login`, `/markets`, `/market/[slug]`, `/hedge`.

3. **30–45 min (Design Basic Pages & Layout):** Draft simple page layouts (wireframe in code). Build a **Login page** (fake auth) that accepts any username/password (since no real auth). After login, redirect to **Markets List**. Create placeholder UI: header/nav, a list of markets, each with question and yes/no prices. Use Tailwind for quick layout and style. On the Markets page, plan a button to go to a market's detail. Add a "Logout" or "Home" CTA for navigation. Keep text clear and action-oriented (e.g. "View Market", "Login").

4. **45–60 min (Fetch Real Market Data):** Write code to call Polymarket's Data API. For example, use `fetch('https://data-api.polymarket.com/markets')` (or their newer Gamma API) to list markets [5]. Use Next.js data fetching (`getStaticProps` for static prefetch or client-side `useEffect`) to populate the Markets list with real questions and prices. Display each market's title, outcomes, and current prices. Confirm CORS access; if needed, use a Next.js API route to proxy API calls. Test that real data appears.

5. **60–90 min (Market Detail Page):** Build the Market detail page (parameterized by slug). Fetch that market's details and recent trades via Polymarket API (e.g. `GET /trades?market=...`) [4]. Show the yes/no prices, and *interactive chart* of price history (using a quick React chart library like Chart.js or Recharts). The chart immediately grabs attention and helps users trust the data [8]. (For now, static chart with one data series is enough.) Add textual stats (volume, last trade price).

6. **90–120 min (Hedging Calculator Logic):** Implement a simple hedging calculation. For example, if a user "bets $X on Yes", the app shows how to buy No shares to create a risk-neutral position. Or allow user to input two correlated markets to hedge between them. Keep it simple: e.g. "To hedge $100 Yes position at price P, you buy $Y of No" and display profit scenarios. The key is showing **numerical output and explanation** clearly. (Cite [19] here to emphasize why hedging matters: "Effective hedges create market liquidity [9]."") Don't integrate real transactions; simulate results in UI.

7. **120–150 min (Multi-step Flow & Fake Auth):** Complete the user flow: after login, Markets list → Market detail → Hedging form. The user enters a bet amount, and the app displays hedge instructions. No real user accounts needed; you can treat the judge as a "test user." Generate a demo email/pass (like `test@example.com / password123`) and display it on login page for the judge. Store it in local storage or mock it in code. Ensure flows are seamless and buttons are labeled in a friendly, reassuring way (using "Buy", "Hedge", "Home").

8. **150–180 min (Styling & Persuasion Elements):** Add **visual cues and persuasive language**. Use consistent, professional colors and fonts (Tailwind can help). For trust, add an SSL padlock icon (or text "Secure"), a brief note like "Using live Polymarket data – your decisions are data-driven" to build credibility [8]. Make CTAs prominent (use a distinctive button style) [8]. Avoid clutter: no pop-ups or hidden info. On forms (e.g. bet entry), use clear labels and reassure "No payment needed" for a demo. Ensure loading spinners appear during data fetch (trust tip: it shows we care about feedback [10]).

9. **180–210 min (Charts and Visual Feedback):** Polish the chart on market detail. Make it responsive and add tooltips on hover. Include a simple legend. Visuals drastically improve comprehension – as in the stock index example below – so our judge can quickly see probability trends【52†】. Ensure numeric values update when inputs change.

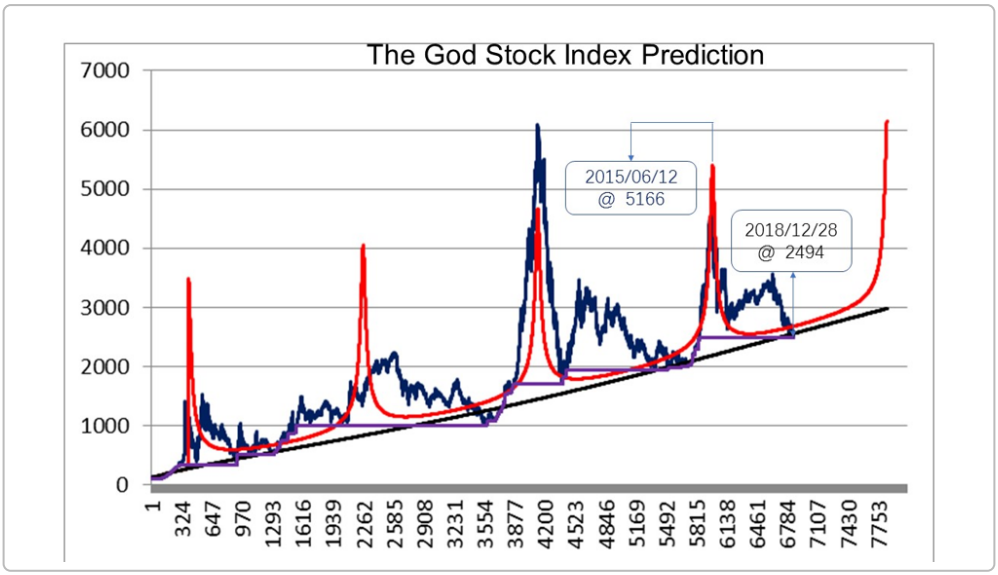

Figure: A stock index "prediction" chart (example). Visualizations like this quickly communicate trend information. In PolyHedg we use dynamic charts of Polymarket odds, making complex data intuitive.

1. **210–240 min (Cut Corners & Prioritize):** By now, we have core features. Skip extras: e.g. social login, wallet, user profile. Focus on refinement: if time is tight, hardcode defaults (e.g. default market ID if needed) or static explanatory text. According to hackathon best practices, avoid scope creep and stick to the original MVP [11].

2. **240–270 min (Responsive Testing & Debug):** Test the app on different screen sizes (mobile vs desktop). Fix obvious bugs. Ensure the hedging calculation is correct (one or two sanity checks). Make the interface mobile-friendly with Tailwind's responsive utilities. If a Polymarket API call fails, show a friendly error ("Could not load data – try refreshing").

3. **270–300 min (Cloudflare Deployment):** Finalize Cloudflare Pages settings. Push the code to main – Cloudflare auto-builds a static site [3]. In the Pages dashboard, set the custom domain `polyhedg.com` (ensure DNS is pointed to Cloudflare per instructions [12]). Get the site live and test on actual domain. HTTPS is automatic. This instant deployment is a huge speed win over manual server setup.

4. **300–330 min (Final UI Polish):** With the site live, refine UI copy and visuals. Add trust signals: e.g. "Powered by Polymarket" logo or citation (social proof), a short "How It Works" blurb. Check color psychology: use blue for CTAs or success (e.g. "Confirm Hedge") [13]. Keep checkout/spinner safe-looking (though no real money is used). Remove any debug consoles and ensure no private keys are visible.

5. **330–360 min (QA with Fresh Eyes):** Do a quick internal demo. Follow the user flow from login to hedging. Verify all links and buttons work. Check that dummy credentials work (login flows). Make minor fixes (typos, alignment). Write down “demo steps” or keep the UI intuitive enough so Oleg won’t need a manual.

6. **360–390 min (Demo Prep):** Record or rehearse a 2-minute pitch: Oleg’s first impression matters. Add concise instructions on the site (e.g. a “Help” text: *“Use the hedging tool by entering an amount and clicking Hedge. This shows how to neutralize risk.”*). Include a hidden link or note (maybe in README on GitHub) with the fake credentials for Oleg to try.

7. **390–420 min (Buffer – Final Testing):** Use this buffer to handle any unexpected issue (API rate-limit, domain glitch, etc.). Review core features. If a feature is shaky, disable it rather than present a bug. Ensure the judge’s path is smooth: login → pick any market → run hedge → see clear result.

8. **420–600 min (Wrap-Up & Polish):** The last hours allow finishing touches. Optimize performance (compress images, minify JS). Check console errors. Update README and commit messages for clarity. If any tasks remain, push them here (e.g. add a favicon, validate HTML). Ensure everything required by judges (the “core features”) is demonstrably working [14] [7].

Throughout the hackathon, we **cut corners wisely**: no user auth backend (just mock it), skip social media, and use libraries instead of custom code. We lean on templates and AI/code examples (e.g. Polymarket API docs) to save time. We also use persuasive UX patterns: **clear CTAs, trust signals, and a friendly tone** [8] [15]. For example, add a reassuring “Live probability data” label, and color positive outcomes green, negative red.

This plan stays laser-focused on the 3 core features (market list, market details, hedging simulator) so that Oleg can navigate and see functionality immediately. By leveraging Next.js’s features and Cloudflare’s fast deploy [1] [3], we maximize development speed and impress the judge with a smooth, credible demo.

Sources: We follow hackathon best practices (planning and MVP focus) [16] [7], use modern dev tools (Next.js for performance [1], Tailwind for styling [2]), pull real prediction data (Polymarket API [4] [5]), and apply UI trust principles (clear CTAs and feedback [8]). This ensures our 10-hour effort yields a polished, persuasive PolyHedg demo.

---

[1] Why use NextJS? - DEV Community
https://dev.to/documatic/why-use-nextjs-mn3

[2] [6] GitHub - HappyHackingSpace/awesome-hackathon: Tools and resources to help you build, design, and win hackathons!
https://github.com/HappyHackingSpace/awesome-hackathon

[3] Get started · Cloudflare Pages docs
https://developers.cloudflare.com/pages/framework-guides/nextjs/deploy-a-static-nextjs-site/

[4] Get trades for a user or markets - Polymarket Documentation
https://docs.polymarket.com/api-reference/core/get-trades-for-a-user-or-markets

5 Get Markets - Polymarket Documentation
https://docs.polymarket.com/developers/gamma-markets-api/get-markets

7 11 How to Host a Hackathon: The MVP Hackathon Process Explained
https://corporate.hackathon.com/articles/how-to-host-a-hackathon-the-mvp-hackathon-process-explained

8 10 13 15 Designing for Trust: UI Patterns That Build Credibility | by Aleksei | Medium
https://medium.com/@Alekseidesign/designing-for-trust-ui-patterns-that-build-credibility-e668e71e8d47

9 A Primer on Prediction Markets - Wharton Initiative on Financial Policy and Regulation
https://wifpr.wharton.upenn.edu/blog/a-primer-on-prediction-markets/

12 Custom domains · Cloudflare Pages docs
https://developers.cloudflare.com/pages/configuration/custom-domains/

14 16 Tips for Planning Your Project - Devpost.com Help Center
https://help.devpost.com/article/125-tips-for-planning-your-project